
Human-Aware Navigation through an Adaptive Proxemic Costmap Layer and Reinforcement Learning

Vigneshwararaj Sathianarayanan^{*}

TDDE05 , AI Robotics, Linköping University, May 2026

^{*}vigsa504@student.liu.se

Abstract

This project investigates human-aware navigation for a TurtleBot4 in the simulated `office_2` world using the ROS 2 Navigation 2 stack. A custom C++ Nav2 costmap layer paints a tiered proxemic cost field around each tracked human; paired with the Regulated Pure Pursuit controller this hand-tuned baseline reaches 50–60% goal completion with a 0.58 to 0.62 m mean minimum human distance. A Soft Actor-Critic (SAC) agent that adapts the proxemic radii on-line is then layered on top. After four reward design iterations the agent matches the baseline on goal completion (50%) while improving every safety metric: collisions drop from 20% to 0%, mean minimum human distance rises from 0.58 m to 0.92 m, and personal-zone intrusion falls from 48.3% to 42.4%, a Pareto improvement within the $N = 10$ evaluation. The remaining gap to a recent reference paper’s reported 89% success is shown to be primarily infrastructural.

Introduction

Ground mobile robots are increasingly deployed in shared human environments such as offices, hospitals and homes. The standard research stack is the Robot Operating System version 2 (ROS 2) [1] together with the Navigation 2 framework (Nav2) [2], which provides a global planner, a local controller, a layered *costmap* (a grid of cost values used by the planner) and a Behaviour Tree recovery system. The TurtleBot4 differential-drive robot with a 360° LiDAR is the canonical test platform.

Operating the bot among people imposes more than collision avoidance: the robot is expected to keep distance, slow down and avoid sudden manoeuvres. Hall’s proxemic framework [3] divides interpersonal space into intimate (≤ 0.45 m), personal (≤ 1.2 m) and social zones (indoor robotics typically uses ≤ 2.0 m for the social zone, narrower than Hall’s original ≤ 3.6 m [3]). There are two technique families that address this objective. The first augments a classical planner with a hand-tuned social cost term; the dominant classical planner is the Dynamic Window Approach (DWA) [4]; a typical example is the Social Force Window planner [5], which adds a Social Force Model (SFM) [6] cost to DWA’s scoring. The second family applies learning, either end-to-end [7] or in a hybrid way that learns to tune a classical planner’s parameters [8] and combine safety with adaptability.

The reference for this project is Martini et al. [9], a preprint that pairs an SFM-augmented DWA with a Soft Actor-Critic (SAC) [10] agent adapting cost weights. It reports 89% goal completion against 77% for a hand-tuned baseline across twelve scenarios. SAC is an off-policy actor-critic algorithm with twin Q-networks (critics), a replay buffer of past transitions, and a maximum-entropy objective. Its convergence theory assumes that the replay transitions are independent and identically distributed (i.i.d.).

This project implements a two-phase system in the `office_2` world. The default world spawns six walking humans; this project overrides the dynamic-objects graph (`sim_overrides/office_2.py`) to spawn twelve walking humans on a 3×3 grid, so the agent sees multiple intersections with varying human density. Roughly a dozen further stationary humans

are seated in cubicles from the static map. Phase 1 introduces a custom C++ Nav2 costmap layer that imposes a proxemic cost field around each tracked human. This is paired with the Regulated Pure Pursuit Controller (RPP) [2], which follows a lookahead point on the global path with cost-regulated velocity scaling. Phase 2 adds a SAC agent that adapts the proxemic distances at runtime, inspired by [9] but applied to costmap-layer parameters rather than DWA weights, which makes the approach controller agnostic.

Hypothesis. *An RL-tuned proxemic policy produces a more socially compliant policy than a hand-tuned baseline without sacrificing task throughput.*

Methods

The system uses the course *TDDE05* provided launcher `air.bringup` for Nav2, the `air_simple_sim` simulator and the `office_2` world. Phase 1 introduces the C++ package `air_social_costmap` and the Phase 2 adds Python modules inside `air_project_nav`, plus training and evaluation scripts.

Data Flow

Figure 1 summarises the architecture. A Python node `human_aware_nav` filters `/semantic_sensor_hf` detections of class `human`, applies a TF2 transform from the sensor optical frame to the global map frame, and republishes positions on `/tracked_humans` (`PoseArray`). The C++ social layer subscribes to this topic and writes proxemic cost into both the local costmap (a 3×3 m rolling window) and the global costmap. In Phase 2 it also subscribes to `/social_params` (`Vector3Stamped`), on which the SAC agent publishes new proxemic radii every 0.5 s.

Phase 1: Proxemic Costmap Layer

The layer is a C++ subclass of `nav2_costmap_2d::Layer` [2] overriding `updateBounds()` (which declares the affected rectangle) and `updateCosts()` (which writes the values). For each human, cells receive a distance based cost according to Table 1. The exponential decay through the personal zone is taken from the proxemic literature: it produces a steep gradient near the intimate boundary and a soft tail, so the planner strongly prefers to maintain > 0.45 m clearance without forbidding the entire personal-space region.

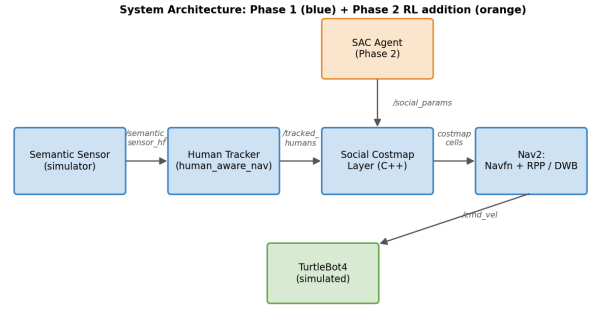


Figure 1: System architecture. Phase 1 (blue) is a four-stage pipeline from the semantic sensor through the human tracker, the proxemic cost field and the Nav2 planner/controller. Phase 2 (orange) adds a SAC agent publishing new proxemic distances on `/social_params` every 0.5 s.

Table 1: Tiered proxemic cost as a function of distance d .

Distance d	Cost value
$d < \text{intimate} (0.45 \text{ m})$	254 (lethal)
$\text{int.} \leq d < \text{pers.}$	$253 e^{-3(d-\text{int.})/(\text{pers.}-\text{int.})}$
$\text{pers.} \leq d < \text{soc.}$	linear, ~ 12 to 0
$d \geq \text{social} (2.0 \text{ m})$	0

Three implementation details proved necessary: (i) Proxemic radii are stored as `std::atomic<double>` so the agent can rewrite them from the ROS 2 executor thread (which dispatches callbacks) while the costmap thread reads them at every cell, lock-free. (ii) The previous cycle’s bounding box is reused in `updateBounds()`, otherwise stale cost lingers at cells from which a human has moved because lower layers do not include those cells. (iii) The transform from the message frame to the layer frame (`odom` local, `map` global) is recomputed each cycle. This is because the local costmap moves itself with the robot’s position. When a cell already has cost from a lower layer, the new value is merged by *max-merge*, so walls (254) dominate proxemic cost and overlapping halos pick the stronger constraint rather than summing into overflow.

Phase 2: Reinforcement Learning Framework

The environment is a class compliant with the `gymnasium` interface, so the Stable-Baselines3 [11] SAC implementation can be used directly. The Markov Decision Process (MDP) is realised as fol-

lows: every `step(action)` publishes the action on `/social_params`, sleeps 0.5 s while Nav2 actively drives, and returns the new observation, reward, and termination flags. An episode ends on goal-reached, Nav2 abort (`nav_failed`), or a 200-step timeout. Collisions ($d_{\text{human}} < 0.22$ m) contribute a per-step penalty but *do not* terminate the episode, because the simulator does not reset the robot between episodes. An earlier version that did terminate on collision trapped the agent in a one-step penalty loop, since the post-collision pose immediately re-triggered collision at the start of the next episode.

State (45-d float vector): (i) Distance and bearing to a local goal placed 2 m ahead of the robot along the published global plan [9, eq. 4]. (ii) For each of the four nearest tracked humans, (dist., bearing, speed, approach-rate) in the robot frame (approach-rate is the component of the human’s velocity along the human-to-robot line, positive iff approaching; velocities come from nearest-neighbour association of successive `/tracked_humans` messages). (iii) The current (intimate, personal, social) triple. (iv) The LiDAR scan min-pooled into 24 angular sectors saturated at 10 m. Missing humans are padded with a far-away sentinel.

Action: (intimate, personal, social) $\in [0.2, 1.5]^3$ m. The env clips, sorts, and publishes (the layer also bounds-checks on receipt).

Reward: a weighted linear combination following [9] with two documented coefficient reductions:

$$r = c_d r_d + c_h r_h + c_v r_v + c_o r_o - c_p r_p - c_s r_s + r_{\text{coll}}. \quad (1)$$

Terms and coefficients are in Table 2. The SFM work term implements [9, eq. 1]: the magnitudes of the net pedestrian and obstacle repulsive forces on the robot plus the reaction forces back on each human (Newton’s third law). Forces follow Helbing–Molnár [6]: $F = A \exp((r_{\text{sum}} - d)/B) \hat{n}$, with pedestrian $(A_p, B_p) = (2.1, 0.3)$ and obstacle $(A_o, B_o) = (10.0, 0.2)$. The deviations from [9] are $c_p = 0.5$ (vs. 2.0) and $r_{\text{coll}} = -50$ (vs. -400). Both reductions were necessary to prevent training divergence in the absence of a clean simulator reset (Section).

Table 2: Reward terms (against the local goal where applicable).

Term	Formula	Coeff.
Progress r_d	$d_{t-1} - d_t$	$c_d = 20.0$
Heading r_h	$1 - 2\sqrt{ \phi /\pi}$	$c_h = 0.4$
Velocity r_v	$(v - v_{\text{max}})/v_{\text{max}}$	$c_v = 2.0$
Clearance r_o	$(d_{\text{min}}^{\text{lid}} - 3)/3$	$c_o = 2.0$
Proxemic r_p	$1/\max(d_{\text{min}}^{\text{hum}}, 0.3)$	$c_p = 0.5$
SFM work r_s	$ F_{\text{ped}} + F_{\text{obs}} + \sum_i F_{r \rightarrow h_i} $	$c_s = 2.5$
Collision	-50 if $d_{\text{min}}^{\text{hum}} < 0.22$	—

Training. SB3 [11] SAC with a two-layer 256-unit fully-connected policy (matching [9, §3.5]), entropy auto-tuning, batch 256, learning rate 3×10^{-4} , replay buffer 10^5 , 500-step random warm-up, one gradient update per env step. Training runs on CPU since CUDA was not available in the course aptainer container.

Episode Sampling and Controller Comparison

Goals are drawn from a fixed waypoint set so static and RL evaluations sample identical scenarios under matched seeds. An initial 10-waypoint set placed six points inside cubicles, requiring tight stop-rotate manoeuvres through ~ 1.5 m doorways that confounded controller comparison. The final set has seven corridor points $(\pm 4, 0)$, $(\pm 8, 0)$, $(0, \pm 4)$ and $(0, 0)$. The proxemic layer is controller-agnostic, so two local controllers are compared with the same layer and same parameters: RPP, which follows the global path with cost-regulated velocity scaling, and the DWB (Dynamic Window B) local planner, the ROS 2 reimplement of DWA [4], which samples a fan of candidate velocities and scores each trajectory using costmap-reading critics. The DWB `BaseObstacle.scale` critic weight was boosted from 0.02 to 0.5 so the social cost actually influences trajectory scoring.

Results

Experimental Setup

All evaluations were performed in `office_2` with the seven-waypoint sampler. Each configuration

was evaluated for $N = 10$ episodes with the random seed sequence replayed across static and RL modes (identical goal sequences from identical starts). Per-episode metrics: **success** (goal within 0.5 m), **collision** ($d_{\min}^{\text{hum}} < 0.22$ m at any point), **nav_fail** (Nav2 aborted), elapsed time, path length, minimum human distance over the episode, and personal-zone intrusion percentage (steps with $d_{\min}^{\text{hum}} < 1.2$ m). In static mode the evaluator publishes the constant triple (0.45, 1.0, 1.4) on `/social_params` every step, slightly tighter than the textbook Hall radii (0.45, 1.2, 2.0) used as the layer’s compile-time defaults, because the wider values produced an unacceptable rate of **nav_failed** on the corridors. The RL agent is therefore compared against the (0.45, 1.0, 1.4) baseline. The simulator does not reset between episodes, so the static baseline’s per-episode metrics depend on the cumulative trajectory; comparisons must be made *within* a single evaluation run. With $N = 10$, the standard error on a 50% success rate is approximately 16 pp, so differences smaller than 15 pp in success rate are not statistically distinguishable.

Iterative Reward Design

Four reward versions were tested. Each iteration was motivated by a failure mode in the previous, itself a finding worth recording.

v1 used action range $[0.1, 5.0]$ m, $\cos \phi$ heading reward, no SFM, terminate-on-collision. SAC inflated the bubble until no path existed: 0% success, 100% **nav_failed**. **v2** tightened the action to $[0.2, 1.5]$, switched to the paper-faithful heading reward $1 - 2\sqrt{|\phi|/\pi}$ [9, eq. 5], added clearance term r_o [9, eq. 7], and made collisions non-terminating. Result: 20% success, 30% collisions, an aggressive squeeze-through policy. **v3** added the SFM work term r_s ($c_s = 2.5$) and human velocity vectors in the state (obs dim $37 \rightarrow 45$): 30% success, 0% collisions, $d_{\min} = 0.74$ m (vs. 0.62 m for paired static). **v4** placed the local goal 2 m ahead on the published global plan [9, eq. 4]; both reward terms r_d, r_h and the observation goal entry now refer to this local goal, and the velocity reward was corrected to $(v - v_{\max})/v_{\max}$. Training reached the highest peak return of all four versions (**ep_rew_mean** = -591 at step 1504, against v3’s -814), confirming that paper-faithful goal information yields a richer learning signal.

Quantitative Results

Table 3 summarises the comparison. All RL rows use SAC trained for 2000–3000 environment steps on CPU. Wall-clock training time ranged from ~ 18 min (v3, 2023 steps) to ~ 44 min (v3 with `--reset-home`, 3414 steps); v4 took ~ 26 min for 2892 steps. Each ten-episode eval added 15–25 min.

Three findings stand out. (i) The v4 agent matches the paired static baseline on goal completion (50% versus 50%) while reducing collisions from 20% to 0%, raising mean minimum human distance from 0.58 m to 0.92 m, and reducing intrusion from 48.3% to 42.4%. Given the $N = 10$ caveat, the success parity is robust (identical outcome), the collision-rate difference is substantial, and the $+0.34$ m $d_{\min}$ gain is well outside noise; intrusion is on the boundary of significance. (ii) The v3 agent (without the local-goal formulation) reaches only 30% success but still produces $d_{\min} = 0.74$ m against 0.62 m for paired static, suggesting that safety behaviour emerges from the SFM reward term while throughput recovery requires the local goal. (iii) The choice of local controller has a first-order effect independent of any RL tuning: with identical proxemic parameters and layer, switching from RPP to DWB raises $d_{\min}$ from 0.62 m to 1.12 m and reduces intrusion from 48.2% to 5.2%, larger than any single RL reward-shaping change. This is explained by their differing response to high cost: RPP follows the global path and crawls through cost gradients via velocity scaling; DWB samples candidate velocities and routes *around* cost via trajectory rejection.

Training Dynamics

Across all four versions, mean episode return rose between steps 500 and 1500, then the critic loss grew roughly 2–3 $\times$ between the peak-reward step and the end of training, with the return stagnating or regressing. For v4, critic loss climbed from 443 (at peak) to 1177 by step 2892; for v3, from 180 to 663. Figure 2 plots representative runs. The v4 peak was -591 at step 1504, degrading to -660 by step 2892. The training script saves only the final model, so evaluation used post-divergence weights; a standard best-model evaluation callback would have captured the peak and is the single highest-impact future-work item.

A separate limitation is that the v1–v4 progression confounds multiple changes per version (for example, v3 introduces both the SFM reward and human velocity in the state), so improvements cannot be

Table 3: Mean per-episode metrics ($N = 10$ episodes per row; identical seed sequences within each static/RL pair).

Configuration	Success %	Coll %	NavFail %	Time (s)	Path (m)	Min hum (m)	Intrusion %
RPP + static (initial eval)	60	0	40	29.7	4.50	0.62	48.2
RPP + RL v1	0	10	100	43.0	1.76	0.17	40.9
RPP + RL v2	20	30	80	38.3	3.31	0.34	69.3
RPP + static (v3 paired)	60	0	40	29.7	4.50	0.62	48.2
RPP + RL v3	30	0	70	38.6	3.48	0.74	49.2
DWB + static	40	0	50	30.7	4.25	1.12	5.2
RPP + static (v4 paired)	50	20	50	24.8	3.81	0.58	48.3
RPP + RL v4	50	0	50	29.6	4.03	0.92	42.4

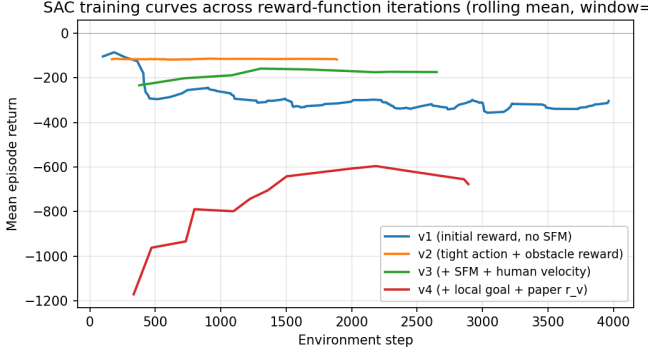


Figure 2: Mean episode return versus environment step for one representative run of each reward-function iteration (rolling mean over three rollouts). Absolute reward values are not comparable across versions because the reward function changes (v3 and v4 add the SFM term), so the figure shows learning *dynamics*: v4 climbs steeply to -591 around step 1500 then degrades, illustrating the critic-loss divergence pattern.

cleanly attributed to a single change; a controlled ablation was not run for time reasons.

Discussion

The v4 Result in Context

The v4 agent represents a Pareto improvement on safety while preserving throughput: not a different operating point on a trade-off curve, but closer to the corner where both safety and throughput are good. The SFM work term acts as soft repulsion across the entire perceptible neighbourhood (not just within the costmap proxemic radii), so the agent maintains a larger spatial buffer than static radii alone can achieve, and the local-goal formulation provides a bounded progress signal at every step. With $N = 10$, only the collision-rate difference and the $+0.34$ m $d_{\min}$ gain are clearly outside noise; the

-5.9 pp intrusion gain is on the boundary.

Comparison with the Reference Paper

Martini et al. [9] report 89% goal completion for their RL agent against this project’s best of 50%. Several architectural differences, in approximate order of importance, contribute to the gap:

1. **Controller and action semantics.** Their agent outputs DWA cost weights; the controller cannot structurally fail because DWA always selects some velocity from its dynamic window. This project’s agent outputs proxemic distances that can produce no-path-exists conditions, explaining the elevated `nav_fail` rate.
2. **Pedestrian simulator.** The reference uses HuNavSim [12], in which humans physically react to the robot via the SFM. The course simulator `air_simple_sim` walks humans on a fixed graph regardless of robot pose, removing bidirectional negotiation.
3. **Episode reset.** The reference resets to a fixed start pose each episode (i.i.d. start distribution). This project cannot, so each episode starts from the previous terminal pose, violating i.i.d. and destabilising SAC’s value-function regression.
4. **Training scale.** SAC for sim navigation typically uses 10^5 – 5×10^5 env steps. CPU at 2 Hz limited this project to ~ 3000 steps, two orders of magnitude less.

The reference paper’s *hand-tuned* baseline (77%) also outperforms this project’s hand-tuned baseline (50–60%), indicating most of the gap is infrastructural rather than algorithmic. [9] is a preprint and the 89% figure has not yet been peer-reviewed.

Limitations and Future Work

The principal limitations are: (1) $N = 10$ does not distinguish success-rate differences within ~ 15 pp; (2) no ablation across reward terms; (3) training divergence combined with the missing best-model callback means reported eval metrics underestimate peak performance; (4) no simulator reset, the structural cause of the i.i.d. violation and training instability; (5) the static cubicle-seated humans are not distinguished from walking pedestrians, inflating effective obstacle density.

The highest-leverage future-work items are: add a Stable-Baselines3 `EvalCallback` for best-model selection (addresses limitation 3 with the smallest change); add a velocity-based filter to `human_aware_nav` to drop long-stationary detections (limitation 5); integrate HuNavSim [12] for reactive pedestrians and a reset service (limitation 4); implement a custom DWB SFM trajectory critic in C++ to match [9]’s controller design; and run a controlled ablation with $N \geq 30$ (limitations 1 and 2).

Conclusion

The Phase 1 hand-tuned proxemic costmap layer with RPP achieved 50–60% goal completion across `office_2` evaluations with 0–20% collisions and a 0.58–0.62 m mean minimum human distance. The Phase 2 SAC agent adapting the proxemic distances at runtime achieved 50% goal completion (matching paired static) with 0% collisions (vs. 20%), $d_{\min} = 0.92$ m (vs. 0.58 m) and reduced intrusion (42.4% vs. 48.3%), a Pareto improvement on safety within $N = 10$. The decisive change was the local-goal formulation [9, eq. 4]. A controlled comparison further showed a first-order controller effect: switching from RPP to DWB raises $d_{\min}$ to 1.12 m and reduces intrusion to 5.2%. The gap to [9]’s reported 89% is primarily infrastructural: a different controller/action space, a reactive simulator, episode reset, and $\sim$ two orders of magnitude more training compute.

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, 2022.
- [2] S. Macenski, F. Martín, R. White, and J. G. Clavero, “The Marathon 2: A Navigation System,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 2718–2725.
- [3] E. T. Hall, *The Hidden Dimension*. New York: Anchor Books, 1966.
- [4] D. Fox, W. Burgard, and S. Thrun, “The dynamic window approach to collision avoidance,” *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.
- [5] N. Pérez-Higueras, R. Otero, F. Caballero, and L. Merino, “Social Force Window Planner,” GitHub repository `robotics-upo/social_force_window_planner`, 2023.
- [6] D. Helbing and P. Molnár, “Social force model for pedestrian dynamics,” *Physical Review E*, vol. 51, no. 5, pp. 4282–4286, 1995.
- [7] Y. F. Chen, M. Everett, M. Liu, and J. P. How, “Socially aware motion planning with deep reinforcement learning,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 1343–1350.
- [8] X. Xiao, Z. Wang, Z. Xu, B. Liu, G. Warrnell, G. Dhamankar, A. Nair, and P. Stone, “APPL: Adaptive Planner Parameter Learning,” *Robotics and Autonomous Systems*, vol. 154, p. 104132, 2022.
- [9] M. Martini, N. Pérez-Higueras, A. Ostuni, M. Chiaberge, F. Caballero, and L. Merino, “Adaptive Social Force Window Planner with Reinforcement Learning,” arXiv preprint arXiv:2404.13678 [cs.RO], April 2024.
- [10] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft Actor-Critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. International Conference on Machine Learning (ICML)*, 2018, pp. 1861–1870.
- [11] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [12] N. Pérez-Higueras, R. Otero, F. Caballero, and L. Merino, “HuNavSim: A ROS 2 human navigation simulator for benchmarking human-aware robot navigation,” *IEEE Robotics and Automation Letters*, vol. 8, no. 11, pp. 7130–7137, November 2023.